QuIM-RAG Intelligence Pro — Concepts From Scratch to Advanced

A Teaching Document

2026-10-23

How to read this document

This is not the technical script (which is a demo narration) and not the equations document (which is a dense reference of 31 formulas). This document is for someone who wants to actually **understand** the system — starting from "what is a document question-answering system" and ending at the specific, novel pieces this project built. Every section builds on the one before it. If a term is used before it's defined, it's defined in an earlier section — read in order the first time through.

Each concept gets three things: **the intuition** (what problem is this solving, in plain language), **the mechanism** (how it actually works, with the real formula), and **how this project uses it** (the actual file and what it actually does, not a hypothetical).

Part 1 — The Problem This Whole System Exists to Solve

1.1 What is "document question answering"?

You have a PDF — a textbook, a manual, a contract — and you want to ask it questions in plain English and get back answers, with proof of where the answer came from. Two bad approaches bookend the design space:

- **Just ask an LLM** ("ChatGPT, what does this manual say about warranty terms?") — but the LLM has never seen your specific PDF, so it either refuses or, worse, confidently makes something up. This made-up-but- confident behavior is called **hallucination**, and it's the central enemy this whole project is built to fight.
- **Just search for keywords** (Ctrl+F) — finds exact word matches but understands nothing: search for "termination clause" and a Ctrl+F misses a paragraph that says "this agreement may be ended by either party" even though that *is* the termination clause.

Retrieval-Augmented Generation (RAG) is the fix: find the specific passages in *your* document that are relevant to the question, hand only those passages to the LLM as context, and ask it to answer using only that context. The LLM never has to "know" anything about your document in advance — it just has to read the handed-over passages and summarize/answer from them, which is a much easier and much more honest task than answering from memory.

question → [RETRIEVE relevant passages from YOUR document] → [LLM reads passages + question] → answer + citations

Everything in this project is either (a) making the RETRIEVE step find the right passages more reliably, or (b) making the system honest about when it isn't sure the passages it found are good enough, or (c) checking, after the fact, that the answer the LLM gave is actually backed by what it was handed.

1.2 Why this specific project exists

This project extends a real, published academic paper: **QuIM-RAG** (Saha, Saha & Malik, 2024, IEEE Access) — "Question-Intent Matching" RAG, which indexes documents not by their raw text but by *questions a chunk of text could answer*, then matches the user's question against that question-index. That paper tested its idea on a private university-catalog dataset we don't have access to. Everything in this project that claims a number either (a) is that paper's own published number, clearly labeled as such, or (b) is our own measurement on a public, substitutable benchmark (OHR-Bench — more on this in Part 4) — the two are never blended or confused, because that would be a form of quiet dishonesty about what was actually tested.

Part 2 — Getting Text Out of a Document (Ingestion)

2.1 The chunking problem

An LLM can't usefully "read" a 200-page PDF as one blob — there's a limit to how much text fits in its context window, and even within that limit, stuffing in everything makes it harder for the model to find the one relevant paragraph among thousands of irrelevant ones (this is sometimes called the "needle in a haystack" problem). So the first thing any RAG system does is **chunking**: cut the document into smaller, overlapping pieces (e.g., 500 words each, with a 50-word overlap between consecutive chunks so an idea that spans a chunk boundary isn't cut in half and lost). Each chunk becomes a separate, independently retrievable unit.

In this project: quimrag/core.py extracts text per page, then splits into overlapping chunks, storing each with its originating document, page number, and (critically, used heavily later) whether the text came from a native PDF text layer or had to be OCR'd, and if OCR'd, how confident the OCR engine was.

2.2 Native text vs. OCR — and why it matters downstream

A PDF can store its text two ways: as actual selectable text (native) or as a picture of text — e.g., a scanned page (image-only). For the second kind, the computer has no idea what the words are until **Optical Character Recognition (OCR)** — software that looks at the picture and guesses the characters — converts it to text. OCR is not perfect: blurry scans, unusual fonts, or low resolution make it guess wrong, and a wrong guess here corrupts everything downstream (you can't retrieve or answer correctly from a sentence that's actually nonsense because OCR misread it).

Why this matters for this project specifically: the real academic benchmark used here, OHR-Bench, exists *specifically* to study this problem — it's built from real documents with real OCR noise at different severities (clean/mild/moderate/severe), with the clean, perfectly-correct version of every page also available as ground truth, so you can measure exactly how much OCR noise degrades a RAG system's accuracy. Section 7 measures exactly this.

pytesseract.image_to_data() (the OCR engine this project uses) doesn't just return guessed text — it returns a **confidence score per word**, which this project averages per page into a single ocr_confidence ∈ [0,1] value and stores alongside the chunk. This number becomes one of the four input signals to EAR (Part 5) — the idea being: if the text a chunk is built from was extracted with low confidence, an answer built from that chunk should be trusted less, automatically, without a human having to notice the scan was blurry.

Part 3 — Finding the Right Passage (Retrieval)

This is the "R" in RAG, and it's the single most important step: if retrieval finds the wrong passage, no amount of good LLM reasoning afterward can save the answer — the model is reasoning over the wrong evidence.

3.1 Lexical retrieval: BM25

Intuition: the simplest thing that could possibly work is keyword matching — but a *smart* version of it, that knows some words are more informative than others ("the" appearing in both the query and a passage tells you nothing; "amortization" appearing in both tells you a lot), and that doesn't just reward a document for repeating a word a thousand times.

Mechanism — BM25 (Okapi BM25, the standard algorithm, from Robertson & Zaragoza 2009) scores a chunk against a query as a sum over the query's words:

score(chunk, query) = Σ_word IDF(word) · [f(word,chunk)·(k1+1)] / [f(word,chunk) + k1·(1 − b + b·|chunk|/avgChunkLen)]

- f(word, chunk) — how many times the word appears in this chunk (term frequency).
- IDF(word) = log((N − n(word) + 0.5) / (n(word) + 0.5) + 1) — **inverse document frequency**: a word that appears in almost every chunk (like "the") gets a near-zero IDF; a rare, specific word gets a high IDF. This is the "smart" part — it automatically figures out which words are informative without anyone hand-labeling stopwords.
- k1 and b are tuning constants controlling how much repeating a word helps (with diminishing returns) and how much long chunks are penalized (a word appearing once in a short chunk is more meaningful than appearing once in a chunk ten times as long).

In this project: quimrag/bm25.py wraps the real rank_bm25.BM25Okapi library (ported in from an earlier, more mature copy of this codebase found during a consolidation pass) — this replaced an earlier, much weaker implementation that was literally just counting overlapping words with no IDF weighting or length normalization at all. The project's own experiments

(Section 5.1 of the paper) show this real BM25 performs essentially identically to dense retrieval alone on this benchmark (Recall@5 0.427 vs. 0.427) — a genuinely measured, reported-as-is result, not adjusted to make either method look better.

3.2 Semantic (dense) retrieval

Intuition: BM25's big weakness is it only matches literal words. If the query says "how much does it cost" and the passage says "the price is", BM25 sees zero word overlap and misses it entirely, even though a human instantly understands they mean the same thing. **Dense retrieval** fixes this by converting both the query and every chunk into a vector of numbers (an **embedding**) such that semantically similar text ends up as *nearby* vectors in that number-space, regardless of the exact words used.

Mechanism: a pretrained neural network (this project uses sentence-transformers/all-MiniLM-L6-v2, a compact model trained specifically to produce good sentence embeddings) maps text to a fixed-length vector (384 numbers, in this model's case). Similarity between two pieces of text is then just **cosine similarity** between their vectors:

$$\text{cosine_similarity}(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$$

This is the dot product of the two vectors, divided by the product of their lengths — it measures the *angle* between the vectors (1.0 = pointing the exact same direction = maximally similar; 0 = perpendicular = unrelated; can go negative for opposite meaning, though in practice sentence embeddings rarely produce strongly negative scores). At query time, the query is embedded once, then compared against every chunk's pre-computed embedding, and the chunks with the highest cosine similarity are returned.

In this project: every chunk's embedding is computed once at ingestion time and kept in memory for fast comparison; MPS/CUDA acceleration (via PyTorch, whichever is available on the machine) is used automatically when present — this is the extent of "GPU optimization" in this project; it is using hardware acceleration already built into the embedding library, not custom-written GPU kernels, and this document says that plainly rather than overstating it as original GPU engineering.

3.3 Combining both: Hybrid retrieval and Reciprocal Rank Fusion (RRF)

Intuition: BM25 and dense retrieval have different, complementary weaknesses (BM25 misses synonyms; dense retrieval can occasionally be fooled by surface-level semantic similarity that isn't actually relevant, and is more sensitive to OCR garbling specific exact terms like part numbers or legal clause names). Combining both rankings tends to be more robust than either alone — this is a very well-established finding in information retrieval, not a novel claim this project makes.

Mechanism — RRF: instead of trying to average two different, incompatible *score scales* (BM25 scores and cosine similarities aren't on the same numeric scale, so averaging them directly is meaningless), RRF only looks at each chunk's *rank* (1st, 2nd, 3rd…) in each ranking, and combines those:

RRF_score(chunk) = Σ_ranking 1 / (k + rank_in_that_ranking(chunk))

k (commonly 60) is a smoothing constant that prevents a chunk ranked 1st in one list and completely absent from the other from dominating too strongly. A chunk that ranks reasonably well in *both* BM25 and dense retrieval ends up with a higher combined RRF score than a chunk that ranks 1st in one but far down (or absent) in the other — this is exactly the "robust to either method's individual mistakes" property being sought.

In this project: quimrag/rrf.py implements this directly; Section 5.1 of the paper's own measured result is that Hybrid-RRF is the only configuration whose improvement over Dense-only survives a corrected statistical significance test (p=0.008) — not assumed to be better because it combines two things, but checked.

3.4 Reranking

Intuition: BM25 and dense retrieval both have to score *every* chunk fast, which means they use relatively cheap, approximate comparison methods. A **reranker** gets to be slower and smarter because it only has to re-score the small shortlist (e.g., top 20) that the fast methods already narrowed things down to — it reads the *query and chunk together* (a "cross-encoder"), rather than comparing two separately-computed vectors, which lets it catch subtler relevance signals.

In this project: cross-encoder/ms-marco-MiniLM-L-6-v2 reranks the Hybrid-RRF shortlist; this is one of the optional configurations compared in the project's own benchmark suite (compare_configs in quimrag/metrics.py).

Part 4 — How Do You Know If Any of This Actually Works? (Evaluation)

4.1 Why you can't just "eyeball" a few answers and call it done

If you check 5 example questions and the answers look good, you've learned almost nothing about whether the system works well on the next 1000 questions — this is the single most common mistake in evaluating AI systems, and the entire reason this project builds a real, repeatable measurement pipeline instead of relying on spot-checks.

4.2 The core retrieval metrics

Given a query with a known correct ("gold") answer location, and a ranked list of chunks the system actually retrieved:

- **Recall@K** = (did any of the top K retrieved chunks contain the gold answer?) — "did we find it at all, somewhere in our shortlist."
- **Precision@K** = (what fraction of the top K retrieved chunks were actually relevant?) — "how much noise did we hand the LLM along with the signal."

- **MRR (Mean Reciprocal Rank)** = average of 1/rank_of_first_correct_result — rewards finding the right answer *near the top*, not just somewhere in the list; a correct answer at rank 1 scores 1.0, at rank 4 scores 0.25.
- **nDCG@K (normalized Discounted Cumulative Gain)** = like Recall but additionally rewards correct results appearing *earlier* in the ranked list, with the reward discounted logarithmically by position — a more complete single-number summary than Recall or Precision alone.

All of these need real ground truth — a labeled answer for "which exact passage, in which exact document, actually answers this question" — which brings us to the dataset question.

4.3 Where does ground truth come from? OHR-Bench

The original QuIM-RAG paper's dataset (22,609 questions over a private NDSU university corpus) has no public release — there is nothing to download, no license terms stated, and the authors' own repository doesn't include it. Using it would mean either fabricating a similar-looking dataset (dishonest — it wouldn't be *that* paper's data) or just asserting numbers with no real evaluation behind them (also dishonest). Neither is acceptable, so a genuine substitute benchmark was needed.

OHR-Bench (Zhang et al., ICCV 2025, a real, peer-reviewed, publicly released benchmark) was chosen because it is the only realistic candidate offering all three properties this project's own novel contribution (EAR, Part 5) specifically needs to be tested against: **real OCR noise** (not a toy simulation), **multi-domain content** (law, finance, textbooks, manuals, papers, news — so results aren't an artifact of one narrow domain), and **page-level ground truth** (so Recall@K/nDCG@K above are actually computable, not guessed). The full comparison against alternatives (BEIR, SQuAD, HotpotQA, Natural Questions, MS MARCO) and why each was rejected is in research/FINAL_VALIDATION/DATASET_SELECTION.md — not repeated here, but it exists and is real, not a post-hoc justification for a choice already made for convenience.

../../../testing/dataset/ in this project's sibling folder structure (see that folder's own README.md) holds the exact, checksummed, git-provenance-verified subset of OHR-Bench this project's benchmark scripts actually read — 262 specific documents out of OHR-Bench's full ~8,500-page release, chosen because that is exactly what the already-computed calibration/evaluation splits reference, not an arbitrarily smaller sample.

4.4 The single most important discipline in this whole evaluation: calibration/evaluation splits

The problem this solves: if you tune a model's settings (like EAR's weights, Part 5) using the same data you then report your headline accuracy number on, your number is inflated — you're measuring how well the model memorized its own tuning set, not how well it generalizes to new questions. This is called **data leakage**, and it is one of the most common ways machine learning results end up looking better than they really are, often by accident rather than intent.

The fix: split your data into two non-overlapping pieces *before* doing anything else — a **calibration** split (used only to fit weights/thresholds) and an **evaluation** split (used only to report the final number, never touched during fitting). This project's own paper has a documented real case study of exactly this mistake happening and being caught: an earlier version of this project's calibration/evaluation split had 51% *document* overlap (the same underlying PDF appeared in both splits, even though individual *questions* were different rows) — which still leaks information, because two questions about the same document aren't truly independent tests. The corrected split enforces **document-disjointness** — zero documents shared between the two splits — and when that fix was applied, EAR's reported improvement (Section 5.2) dropped from a confident, statistically significant +7.1 points to a smaller, no-longer-significant +6.1 points. Reporting that drop honestly, rather than keeping the better-looking leaky number, is the single most important integrity decision in this project's entire research pipeline.

4.5 Statistical significance, confidence intervals, and p-values — in plain language

A single number (e.g., "Hybrid got 44.4% Recall@5, Dense got 42.7%") doesn't tell you whether that difference is real or just noise from which specific 247 questions happened to be sampled. Three tools fix this:

- **Bootstrap confidence interval**: resample your actual results (with replacement) thousands of times, recompute the metric each time, and look at the spread of results you get. If you resampled the same real data a thousand different ways and the metric difference between two systems is consistently positive, you have a real signal; if it flips sign depending on the resample, it's noise. This project reports a "95% CI" — the range that captured 95% of those resampled outcomes.
- **p-value**: roughly, "if there were actually no real difference between the two systems, how surprising would it be to see a difference this large just by chance?" A small p-value (conventionally < 0.05) means "quite surprising" — evidence of a real effect. This project explicitly uses a corrected threshold (0.025, via Bonferroni correction) when checking multiple comparisons at once, because checking many things at the uncorrected 0.05 threshold makes finding at least one false positive likely by chance alone.
- **Power analysis**: the honest companion to a "not significant" result. A non-significant p-value can mean either "there's truly no effect" or "there might be a real effect, but we didn't collect enough data to be sure." A power analysis computes the smallest effect size your sample size *could* have reliably detected — this project's own power analysis (Part 6) found its evaluation could only reliably detect an effect about four times larger than what was actually observed, meaning the "not significant" result is honestly reported as "underpowered," not as "proven to be zero effect" — a real, meaningful distinction this project does not blur.

All of the above (plus McNemar's test, Cohen's h, ROC-AUC, Brier score, Expected Calibration Error, and the Wilson interval) are used throughout this project's validation, each with the

exact formula, spelled out in full in testing/comparison_and_validation/VALIDATION_AND_COMPARISON.md.

Part 5 — This Project's Actual Novel Contribution: Evidence-Adaptive Retrieval & Verification (EAR)

Everything up to this point (chunking, BM25, dense retrieval, RRF, reranking, the evaluation methodology) is **reused, well-established technique**, correctly implemented and honestly measured, but not new. EAR is this project's own idea.

5.1 The problem EAR solves

A RAG system that always answers, no matter how bad the retrieved evidence is, will confidently hallucinate on the questions its retrieval step failed on — exactly as dangerous as having no retrieval check at all. What's needed is a way for the system to recognize, *before* generating an answer, "the evidence I found for this particular question looks weak — I should say so, or look harder, rather than guessing."

5.2 The four signals

EAR computes four numbers for every query, each designed to be computable **for free** — reusing results already computed by the retrieval step, with no extra model calls:

1. **Agreement** — do the BM25-only ranking and the Dense-only ranking of top chunks actually overlap (Jaccard similarity of their top-K chunk-ID sets)? If two very different retrieval methods agree on what's relevant, that's a vote of confidence; if they disagree wildly, it's a red flag that the question might be ambiguous or poorly covered by the document.
2. **OCR confidence** — the mean OCR confidence (Part 2.2) across the retrieved evidence chunks. Low-confidence OCR means the "evidence" might literally be misread text.
3. **Contradiction** — do the top retrieved chunks actually agree with each other (checked via embedding similarity among them, plus a cheap regex-based numeric/date-conflict heuristic)? If the top chunks seem to contradict each other, that's a sign the retrieval didn't converge on one clear, consistent answer.
4. **Complexity** — a heuristic (query length, question-word count, conjunction count) estimating whether this looks like a simple factual question or a harder, multi-part one, which independently matters for how much to trust a single retrieved chunk.

5.3 Combining the signals: logistic regression

Why not just add the four numbers together with made-up weights? Because arbitrary weights are a guess, and a guess that happens to look reasonable isn't the same as a weighting that's actually been shown, on real held-out data, to predict real outcomes. Instead, EAR

learns its weights by logistic regression — the same well-established statistical method used throughout data science for binary prediction problems:

$$\text{sufficiency_score} = \sigma(w_1 \cdot \text{agreement} + w_2 \cdot \text{ocr_confidence} + w_3 \cdot \text{contradiction} + w_4 \cdot \text{complexity} + b)$$
$$\sigma(z) = 1 / (1 + e^{-z}) \quad \leftarrow \text{the logistic ("sigmoid") function, squashes any real number into (0, 1)}$$

The weights $w_1..w_4$ and bias b are fit by finding the values that best predict a real, observable outcome on the **calibration split only** (Part 4.4): "was the top-1 retrieved chunk actually the gold evidence page for this question?" This is what makes the weights *calibrated*, not guessed — and because fitting only touches the calibration split, the evaluation split's headline number is never contaminated by information from its own test.

5.4 Turning a score into a decision

A single probability number isn't directly actionable — EAR converts it into one of four decisions (answer / expand_retrieval / inspect_structure / abstain) via two thresholds, also chosen on the calibration split, not eyeballed: scores above the higher threshold ⟶ answer directly; scores below the lower threshold ⟶ abstain (tell the user "I'm not confident enough in the evidence to answer"); in between ⟶ try to get more/better evidence first.

5.5 What this project found, honestly, when it actually tested EAR

The headline result (full detail in Part 6 and the paper's Section 5.2): on the corrected, leakage-free split, EAR's accuracy-on-answered-questions is 68.4% vs. a 62.3% baseline when it chooses to answer — a real improvement in direction, but with a 95% confidence interval of [−8.3, +19.7] that crosses zero, meaning this particular experiment cannot statistically rule out "no real effect." The honest companion finding (power analysis) is that this specific sample size (38 answered queries) was simply too small to reliably detect an effect of the size actually observed — a limitation of the *experiment*, stated plainly, not a disguised failure of the *idea*.

A second honest finding: the ocr_confidence signal turned out to be **constant across every query in this particular evaluation split** (because, on this run, all evaluated documents happened to be extracted with uniformly high native-text confidence — no severely-degraded pages landed in this specific sample), which makes its fitted weight mathematically forced toward zero, regardless of whether OCR confidence would matter on a different sample. This is reported, not hidden, and is exactly the kind of finding that led directly to EAR v2 (Part 6).

Part 6 — Taking the Diagnosis Seriously: EAR v2 and the Full Validation Suite

6.1 Signal-level diagnostics — is the *score* informative, even if the *threshold* isn't significant?

A single headline coverage/accuracy number (Part 5.5) answers only "did this one specific threshold choice work." It cannot answer a separate, equally important question: "is the

underlying sufficiency score, as a *continuous ranking* of how trustworthy each answer is, actually informative at all — independent of exactly where the threshold line was drawn?" This project adds four further diagnostics specifically to answer that second question (all computed in research/PUBLICATION_PACKAGE/compute_advanced_stats.py, real formulas in testing/comparison_and_validation/VALIDATION_AND_COMPARISON.md Part 3):

- **ROC-AUC = 0.6735** — meaningfully above the 0.5 "random guessing" baseline: EAR's score genuinely does rank more-likely-correct answers above less-likely-correct ones, more often than chance.
- **AURC (Area Under the Risk-Coverage curve) = 0.2869** vs. a full-coverage baseline risk of 0.3765 — using EAR's score to decide which questions to answer first genuinely reduces error rate at matched coverage levels.
- **Brier score = 0.2117** vs. a naive always-predict-the-base-rate score of 0.2348 — EAR's probability estimates are measurably better calibrated than just assuming every answer has the same average correctness rate.
- **Expected Calibration Error = 0.0961** — not tiny, which is exactly what motivated the fix below.

6.2 EAR v2: acting on the diagnosis, not inventing a new idea

Two concrete problems were found above (the dead ocr_confidence signal, and — visible in the reliability diagram, not described here in text — a reliability curve where the *highest*-confidence bin was actually *less* reliable than two bins below it, something a straight-line logistic model can't represent at all, because a sigmoid of a linear combination is forced to be monotonic). EAR v2 fixes exactly these two things and nothing else:

1. **Drop the dead signal** — refit using only the three signals that actually varied (agreement, contradiction, complexity). Fewer free parameters means less overfitting risk on a still-small (n=173) calibration set, and removes a signal whose weight was mathematically forced near zero on this sample regardless of its true value.
2. **Isotonic recalibration** — instead of forcing the probability-vs-correctness relationship through a straight (monotonic-by-construction) sigmoid, fit an arbitrary non-decreasing step function (via the pool-adjacent-violators algorithm) that *can* represent a non-monotonic true relationship, using only the calibration split, then apply it frozen to the evaluation split.

Stated in advance, before running the result (the project's own frozen-protocol discipline, applied here too): these two fixes target *calibration quality* specifically — how well the probability reflects true likelihood — not the amount of information available or the sample size, so they were explicitly **not** expected to flip the non-significant coverage/accuracy result into a significant one. **What actually happened** confirmed exactly that prediction: ROC-AUC stayed essentially flat (0.6735 → 0.6724 — expected, since isotonic regression is a monotonic transform of the same underlying score, so it can't change the ranking, only the calibration of it), while Expected Calibration Error nearly halved (0.0961 → 0.0459) and Brier score improved slightly (0.2117 → 0.2080). A prediction made before the experiment, and confirmed rather than contradicted by the actual result, is itself a small piece of evidence that the diagnosis-

driven methodology here is sound — not just a post-hoc story fit to whatever numbers came out.

6.3 How EAR compares to a real, recent published system

A genuinely comparable, contemporaneous paper — **AB-RAG** (Kamthan, 2026) — computes a conceptually similar weighted-signal confidence score for RAG, evaluated with the exact same AUROC definition this project uses. Their own reported range across four backbone/dataset combinations is 0.600–0.733; this project's 0.6735 sits inside that range, not above all of it and not below all of it — reported exactly as measured. The full analysis, including where this project's zero-added-inference-cost design is a genuine (not spun) advantage and where AB-RAG's best configuration genuinely outperforms this project's result, is in testing/comparison_and_validation/VALIDATION_AND_COMPARISON.md — not duplicated here, but this is the honest, two-sided comparison this document promised in its opening paragraph.

Part 7 — Residual Vector Quantization (RVQ): Compression, and Its Real Cost

7.1 The problem

Every chunk's dense embedding (Part 3.2) is 384 floating-point numbers. For a large document collection, storing and comparing all of these exactly gets expensive in both memory and search time. **Vector quantization** trades some accuracy for a large reduction in both.

7.2 How Residual Vector Quantization works

Stage 1: run k-means clustering (grouping similar vectors together and finding each group's center) on all the raw embedding vectors, with (in this project) 256 clusters. Replace each vector with just the ID of its nearest cluster center — a huge compression, but a lossy one, since all vectors in the same cluster are now indistinguishable from each other.

Stage 2 (the "residual" part): for each vector, compute the **residual** — the leftover difference between the original vector and its stage-1 cluster center — then run a *second* k-means clustering, this time on those residuals. Now each vector is represented by two small cluster IDs (stage-1 center + stage-2 residual center) instead of 384 full floating-point numbers, but reconstructing an approximation of the original vector ($\text{center}_1 + \text{center}_2$) is more accurate than stage-1 alone, because the second stage captures some of what the first stage missed.

compression_ratio = (original bytes per vector) / (bytes needed to store the two cluster IDs)
reconstruction_error = mean squared error between original vector and ($\text{center}_1 + \text{center}_2$)

7.3 What this project actually found — a real, not-glossed-over tradeoff

Measured directly (research/run_rvq_analysis.py, numbers in testing/metrics/): a 768x compression ratio, with a mean reconstruction cosine similarity of 0.761 to the original vectors — sounds

good in isolation. But measuring the *actual retrieval consequence* tells a more complete story: Recall@5 drops from 0.378 (exact vectors) to 0.161 (quantized) — a severe, not a mild, degradation — and specifically, **18.5% of gold evidence chunks are lost entirely** to quantization error (confirmed with a Welch's t-test, $t=-4.71$, $p=3.5\times10^{-6}$, that lost chunks have measurably worse reconstruction fidelity than retained ones — this isn't random noise, it's a systematic weakness). A partial fix — using the compressed representation only to generate a candidate shortlist, then re-scoring that shortlist with the exact, uncompressed vectors — recovers much of the gap (Recall@5 back up to 0.301) while keeping most of the latency benefit. **This project reports RVQ's real limitation plainly rather than only its flattering compression-ratio number** — a 768x compression number alone, without the recall-gap number next to it, would be a misleading, incomplete picture of what RVQ actually costs.

Part 8 — Checking the Answer After It's Generated: EICV-RAG

Everything so far improves what evidence gets handed to the LLM, or decides whether to trust the retrieval step enough to answer at all. EICV-RAG (Evidence Integrity & Counterfactual Verification) is a different, later check: **given an answer the LLM already generated, is that specific answer actually backed by the evidence it was handed?**

8.1 Claim-level attribution

An LLM's answer is first broken into individual atomic claims (e.g., "The warranty lasts 2 years" and "registration must happen within 30 days" are two separate claims, even if they appeared in the same sentence). Each claim is then independently checked against the actual retrieved evidence chunks: *which* chunk(s), if any, actually support this specific claim? This is deliberately a stricter check than just verifying a citation marker (like [C12]) points to a real chunk — a citation can point to a real chunk that simply doesn't say what the claim says it says, and only an independent re-derivation of support (not just "does the pointer resolve") catches that.

8.2 Counterfactual testing — the "would this claim survive without its evidence?" check

For a claim judged SUPPORTED, its supporting chunk is deliberately removed from the evidence set, and the support-checking step is re-run. If the claim now flips to UNSUPPORTED, that's a real, mechanistic demonstration that the claim genuinely depended on that specific evidence — not just that the evidence happened to be present. (If support doesn't change, that's reported honestly as *ambiguous* — it could mean the claim is genuinely redundantly supported elsewhere, or it could mean the model is drawing on its own general prior knowledge rather than the document at all — the system does not claim to distinguish these two cases, because it genuinely cannot without further, not-yet-built tests.)

8.3 What's real here, and what's intentionally not built yet

This project implements one of seven possible counterfactual test types the original design brief specified (evidence deletion); the other six (masking specific spans, replacing evidence with a distractor, injecting OCR noise into otherwise-good evidence, substituting a citation, redundant-evidence controls, and others) are explicitly named as future work, not silently skipped — the full scope-vs-built breakdown is in docs/production/EICV_RAG.md and reproduced as a table in the paper's Section 3.8.

Part 9 — Turning a Research Prototype Into Something You Can Actually Run

9.1 Why "it works in my Streamlit notebook" isn't the same as "it's a real system"

A research script that only runs correctly when you execute cells in exactly the right order, in exactly one person's environment, isn't something anyone else can verify or use. This project's production pass built a real client-server architecture: a FastAPI backend exposing a real HTTP API (/upload, /ask, /verify, /documents), with documents and query history persisted to a real SQLite database (so the system's state survives a restart — and, as described in Part 8 and the paper, a real bug in exactly this restart-survival guarantee was found and fixed, not merely assumed to work), and a separate frontend (a single-page chat interface) that talks to that API over the network rather than being glued directly into the same process.

9.2 Making the whole project genuinely reproducible

The final step was making this exact project folder — the one this document lives in — fully self-contained: its own Python virtual environment (.venv/, built fresh from requirements.txt, not copied from another machine's environment, so it genuinely reflects what's needed to run this code from scratch), its own git history, and a push to a real public GitHub repository. "Deployed," honestly, means exactly this: anyone can clone the repository, create the virtual environment, run the backend and frontend locally, and get the same system described in this document — not a live, permanently-hosted public URL (a free-tier host realistically cannot run the local LLM, Ollama, this system depends on; that constraint is stated plainly rather than hidden behind a demo that would quietly fail for a real visitor).

Closing note

Every specific number in this document is traceable to a real file this project actually generated — either research/experiments/*.json, research/PUBLICATION_PACKAGE/*.json, or the fresh reproduction run in testing/metrics/. Where a result was weaker than hoped (the leaky-split correction, EAR's non-significant CI, RVQ's recall gap, EAR's position inside AB-RAG's range

rather than above all of it), this document reports it exactly that way, on the view that an accurate account of a real, honestly-limited system is more valuable — and more defensible, if anyone ever checks — than a flattering account of an imaginary, better one.